eedi-mining-misconceptions-in-mathematics

8位 Solution

Rank	8
Team	Yannan Chen
Author	Yannan Chen
Topic ID	551412
Version	Japanese Translation

Source: https://www.kaggle.com/competitions/eedi-mining-misconceptions-in-mathematics/discussion/551412
Retrieved with Kaggle CLI on 2026-10-21.

Kaggle Discussionの主投稿と、技術的補足を含む選定済みQ&Aを日本語へ翻訳した版です。code、URL、model名、識別子、数値は原文を保持しています。

上位解法を読み、他チームの献身的な取り組み、学習と推論の細部への細やかな注意、そして非常に洗練されたprompt engineeringの手法に心から感銘を受けました。初心者として、この最終順位を達成できたことを幸運に感じており、謙虚な気持ちで私の解法を共有します。私の解法はシンプルではありますが、それでも効果的で、比較的計算効率が良いと考えています。使用したのは14Bと32Bのmodelのみで、合計の実行時間は約6時間です。

コンペ期間中およびコンペ後にコードや知見を共有してくださったすべての方に感謝を申し上げます。

2つの重要な参考文献

- @sayoulala : https://www.kaggle.com/competitions/eedi-mining-misconceptions-in-mathematics/discussion/543519
  - LoRA finetuning、hard negative mining、recall+rerankの学習スキームについて教えてくれました。
- @cdeotte : https://www.kaggle.com/competitions/lmsys-chatbot-arena/discussion/521294
  - modelの量子化、wise-FTによるLoRA adaptation、VLLMによる推論について示唆に富むアイデアを得ました。

推論pipeline

retrievers（4つのQwen2.5-14b modelをweighted sumでensemble）→ 40 misconceptions → reranker（単一のQwen2.5-32b model）→ 25 misconceptions

学習の詳細

全体として、反復的なhard negative miningが非常に有効だと感じています。

- chatgptでfew-shots promptを使い、追加の学習データを生成しました:
  - 欠けているmisconceptionを埋めるため、983件の合成データサンプルを追加しました。
  - 学習データは `"QuestionId"` で5つのCV-foldに分割しています。各foldについて、valid foldに含まれるunique misconceptionをカバーする追加サンプルを学習用に生成し直しました。
- Retriever（Qwen2.5-14b）: FEATURE_EXTRACTION taskとして、bnb bit4 + LoRAでfinetuningしました。
  - 2台のA100で学習しました。
  - lora parameters: `r=32, alpha=64, target_modules=['q_proj','k_proj','v_proj','o_proj','gate_proj','up_proj','down_proj']`
  - training paramters: `train_group_size=4, effective_batch_size=64, epoch=7, lr_scheduler_type="cosine"`;
  - 5つのCV-foldで学習しました。
    - 最初の4〜5 epochは生成データなしで学習し、map25の改善に注力しました。
    - 最後の2〜3 epochでは生成データを学習に追加しました。これによりmap25とrecall25がさらに改善しました。

    - valid score（map25 & recall25）の改善が止まったときは、次の方法で手動で介入しました:
        - learning_rateを5e-4から1e-5まで徐々に下げる
        - negative data poolのサイズを80から4まで減らし、最も難しいmisconceptionの識別により集中させる
- Reranker（Qwen2.5-32b）: CASUAL_ML taskとして、bnb bit4 + LoRAでfinetuningし、各query–misconceptionペアについて「Yes」のlogitを出力させました。
    - 2台のA100で学習しました。
    - lora parameters: `r=32, alpha=64, target_modules=['q_proj','k_proj','v_proj','o_proj','gate_proj','up_proj','down_proj','down_proj','lm_head']`
    - training paramters: `train_group_size=4, effective_batch_size=64, epoch=4, learning_rate=5e-4, lr_scheduler_type="WarmupDecayLR";`
    - rerankerの学習にはvalidationなしでdataset全体を使いました。Negative dataは4つのretriever（weighted sumでemsemble）により、negative_count = 40で生成しました。

量子化と推論の詳細

- Retrieving:
    - 4つのretriever（5-fold CVの中からLB scoreに基づいて選択）でそれぞれretrieveを行い、weighted sumでensembleしました。
    - bnb bit4+LoRAで推論しています（mergeは不要）。これにより各retrieverが1枚のT4 gpuに収まり、4つのretrieverを2枚のT4で2回実行するだけで済みます。
- Reranking:
    - まずmodelをlora adaptorとmergeします。その際、wise-FTのhyper parameterはalpha=0.8としました。
    - 次にmodelをAWQに量子化します（学習データからランダムに選んだ128サンプルでcalibration）。
    - 最終的な推論にはVLLMを使用しました。各queryをretrieveされた40個のmisconceptionとペアにし、rerankerがscoreを付けるための40個のqueryを生成します。推論を高速化するため、"enable_prefix_caching"を有効にしています。

主な改善点（Public LB）

- 合成データの追加: 単一retrieverのpublic LB scoreが0.3〜 → 0.49
- retrieverのensemble: 0.49 → 0.51
- rerankerの追加: 0.51 → 0.57
- rerankerのLoRA mergeにWise-FTを適用: 0.57 → 0.59

個人的な振り返り

LoRA finetuningから量子化に至るまで、pipeline全体が私にとって非常に新しいもので、何度も諦めかけました。まずはQwen2.5-0.5Bで小さく始めて動作するプロセスを確立しようとしましたが、AWQ量子化したmodelが推論中にランダムな文字を出力し続けました。散々苦しんだ末に、autoawqがQwen2.5-0.5Bのim-head-embed layer

のparameterをexportしていないことに気づき、手動で追加せざるを得ませんでした。それが可能であることを示した@cdeotteの論文がなければ、完全に投げ出していたかもしれません。その安心感のおかげで粘り強く取り組み、最終的にpipelineをスムーズに動かせるようになりました。

コード

- 推論コード: https://www.kaggle.com/code/yannan90/eedi-recall-rerank-lb-0-601-pb-0-584
- 学習コード: https://github.com/yannan90/kaggle-2024-Eedi-8th-place-training-code

補足Q&A

Nikhil Tumbde — 質問

解法の共有をありがとうございます。enable_prefix_caching parameterとwise-FTは初めて知りました。この分野は私にとって非常に新しいため、いくつか質問させてください。理解を助けていただけるととてもありがたいです（できれば次のwriteupにもこれらのポイントを含めてください）。

1. 合成データ生成に使ったprompt（またはその考え方）を共有していただけますか。また、学習データに出現しないmisconceptionについてもデータを生成しましたか。「欠けているmisconceptionを埋めるため、983件の合成データサンプルを追加しました。」
2. 次の点を手動で行う方法も共有していただけますか。私も同じことで苦労しました。

> 散々苦しんだ末に、autoawqがQwen2.5-0.5Bのim-head-embed layerのparameterをexportしていないことに気づき、手動で追加せざるを得ませんでした。>

Yannan Chen — 回答

1. はい、学習データに出現しないmisconceptionについてもサンプルを生成しました。OOF学習を行う際、validationに使うfoldにもいくつかunique misconceptionが含まれるため、それらについても生成しました。使用したpromptは私の母語で作成したものですが、主要な構成要素は次のとおりです:

- タスクと、AIに生成させたい各新規データサンプルのformat（ConstructName、SubjectName、QuestionText、CorrectAnswerText…）を説明する。
- 既存のtrain dataからランダムに選んだ3〜4個のexampleを例として与える。
- AIに生成の基にさせたい新しいmisconceptionを与える（1つでも、リストでも構いません）。
- misconceptionを漏らさないこと、適切なformatで返すこと、その他追加したい制約をAIに念押しする。

Yannan Chen — 回答

1. これは非常に奇妙な問題です。params fileを手動で調整する方法は次のとおりです:

safetensors fileは、lib `from safetensors.torch import safe_open, load_file, save_file` を使えば単純な辞書形式として読み込めます。keyは各layerの名前で、valueはtensorです。

そのため、fileを読み込んだ後は、非量子化modelのparams（同じく辞書として読み込む）と比較し、keyに不一致がないか確認するだけです。不一致があれば、`quant_params[k] = unquant_params.get(k)` のようなコードで辞書を修正します。最後に、修正したparamsを保存し直します。